Banco de Dados

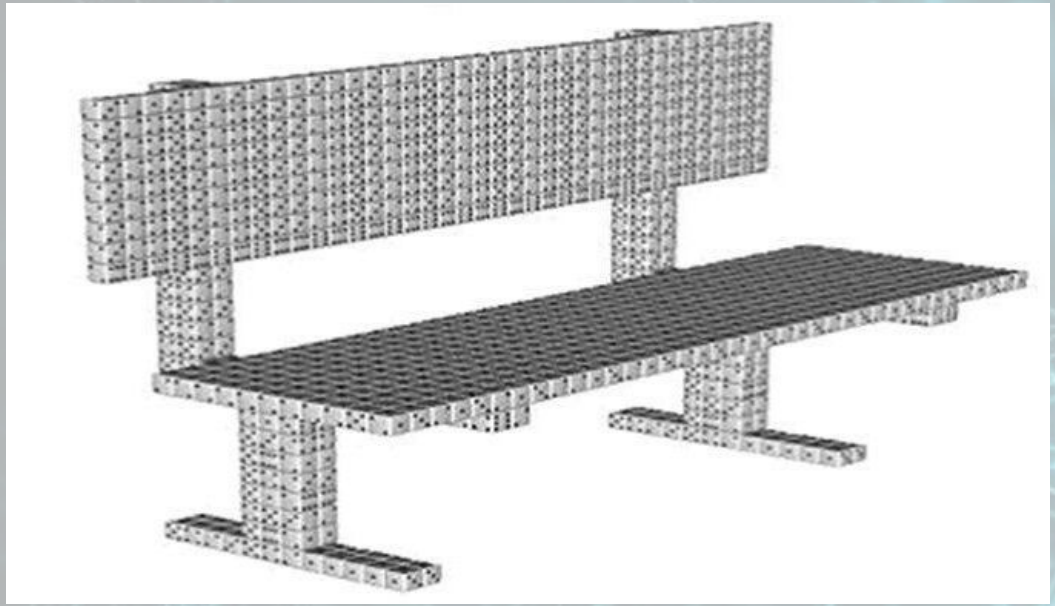
Prof. Ramon Ferreira

Módulo 1:

Design e Modelagem

Tópico 1.1: O que é um Banco de Dados?

Conceito: O que é um Banco de Dados(Database)?



Armazenamento Caótico x Catalogado

Entidade e Atributo

- Entidade: É o substantivo (A Tabela). Ex: Usuário, Produto.

- Atributo: É a característica (A Coluna). Ex: Nome, Data de Nascimento.

Entidade e Atributo

- Qual o nome da Tabela?

- Qual o nome do(s) atributo(s)?

Clientes : Tabela

id	nome	fone
1	João Pedro	2365-6589
2	Guilherme Silva	2365-9863
3	Marcos Luiz	2365-2568
4	Ariadne Peres	2365-1257
5	Vilma Soares	2365-3259
6	Ana Garcia	2365-1567
7	Luiz Augusto	2365-2589
8	André Luiz	2365-9875
9	Pedro Guilherme	2365-1212
10	Anderson Pinto	2365-2626
utoNumeração)		

Registro: 10 de 10

Atividade

Pegue como exemplo a rede social Instagram. Para realizar a publicação de uma foto supomos que é criado um registro na tabela “post”, pegue uma folha e caneta e liste quais atributos essa tabela tem.

Tópico 1.2: Relacionamentos

Chave Primária (Primary Key - PK)

É o CPF/RG do registro. Não se repete.

Identifica exclusivamente cada registro em uma tabela.

Pode ser Natural ou Substituta.

Chave Primária (PK)

Chave PK Natural:

É um dado que já existe no mundo real e que identifica o registro de forma única.

Exemplos: CPF, ISBN (para livros) ou o código de rastreio de um pacote.

- **Vantagem:** Você não precisa criar uma coluna "artificial" extra.
- **Desvantagem:** Se o dado mudar no mundo real (ex: o governo muda o formato do CPF), a manutenção no banco de dados é um pesadelo, pois a PK é usada para relacionar todas as outras tabelas.

Chave Primária (PK)

Chave Substituta (Surrogate Key):

É um valor sem significado de negócio, criado apenas para fins de banco de dados. É aqui que entra o famoso ID (geralmente um número sequencial ou um UUID).

- **Vantagem:** É estável, curta e nunca muda, independente do que aconteça com os dados do cliente.
- **Vantagem Técnica:** Índices numéricos (inteiros) são processados muito mais rápido pelo motor do banco de dados do que textos longos.

Chave Estrangeira (Foreign Key - FK)

Se a Chave Primária (PK) serve para identificar quem é quem dentro de uma tabela, a Chave Estrangeira (FK - Foreign Key) serve para criar a "ponte" entre duas tabelas.

Em termos simples: uma FK é uma coluna em uma tabela que "aponta" para a PK de outra tabela.

Chave Estrangeira (FK)

TABELA CLIENTES

ID_CLIENTE	NOME
1	DANIEL
2	JOAO
3	PAULO

TABELA VENDAS

IDCLIENTE	PRODUTO	QUANTIDADE
1	AGUA	1
1	BISCOITO	2
1	CHOCOLATE	1
2	AGUA	2

PK

FK

Na tabela **Clientes**, o **id_cliente** é a Chave Primária.

Na tabela **Vendas**, você precisa saber qual cliente fez aquela compra. Então, você cria uma coluna chamada **idcliente** (ou **clienteid**) dentro da tabela de Vendas.

Essa coluna na tabela de **Pedidos** é a **Chave Estrangeira**.

Chave Primária
&
Chave Estrangeira

Chave primária vs. Chave estrangeeira

Chave Primária

- Não se repete
- Não pode ser nula

Ex: ID_Cliente em clientes

Chave Estrangeira

- Pode se repetir
- Pode ter valores iguais

Ex: ID_Cliente em pedidos

Summary	Pode repetir?	Pode ser nula?
Primária	✗	✗
Estrangeira	✓	✓

Cardinalidade

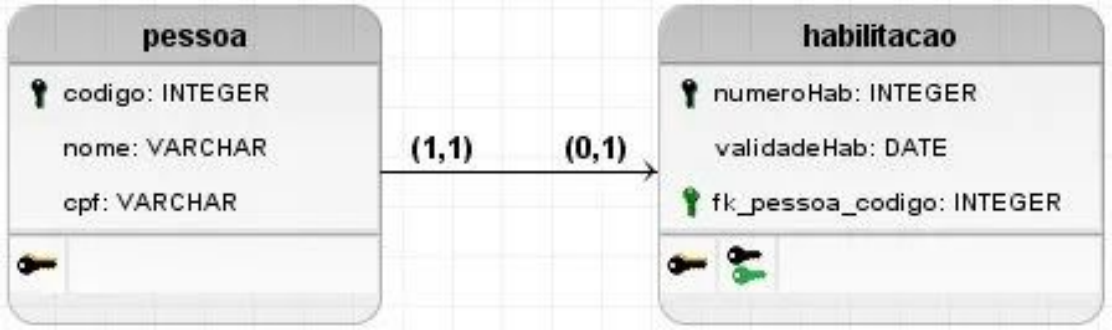
1. **Um para um (1:1):** Um CPF pertence a uma pessoa. (Raro)
2. **Um para muitos (1:N):** Um cliente faz muitos pedidos. (O mais comum)
3. **Muitos para Muitos (N:N):** Muitos alunos estão em muitas disciplinas. (Gera uma tabela "meio-campo").

Cardinalidade

Um para um (1:1):

Neste modelo, um registro na Tabela A pode estar relacionado a apenas um registro na Tabela B, e vice-versa.

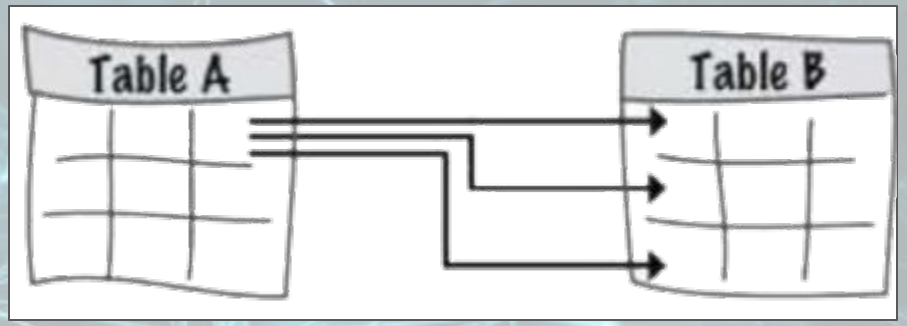
- **Exemplo:** Pessoa e Habilitação. Cada pessoa tem apenas um CNH, e cada CNH pertence a apenas uma pessoa.
- **Uso:** Geralmente usado para separar dados sensíveis ou para dividir tabelas com colunas demais por questões de performance.



Cardinalidade

Um para muitos (1:N):

Este é o tipo mais comum. Um registro na Tabela A pode se relacionar com vários na Tabela B, mas o registro na Tabela B só pode pertencer a um na Tabela A.



- **Exemplo:** Cliente e Pedidos. Um cliente pode fazer 50 pedidos, mas cada pedido específico pertence a apenas um único cliente.
- **Onde fica a FK?** A Chave Estrangeira sempre fica no lado "Muitos" (na tabela de Pedidos, guardamos o ID do Cliente).

Cardinalidade

Muitos para Muitos (N:N):

Aqui, vários registros na Tabela A podem se relacionar com vários na Tabela B.

- **Exemplo:** Alunos e Cursos. Um aluno pode se matricular em vários cursos, e um curso pode ter centenas de alunos matriculados.
- **A "Tabela de Ligação":** Bancos de dados relacionais não conseguem criar um N:N direto. Para resolver isso, criamos uma terceira tabela (tabela intermediária ou associativa) que guarda os IDs de ambos os lados.

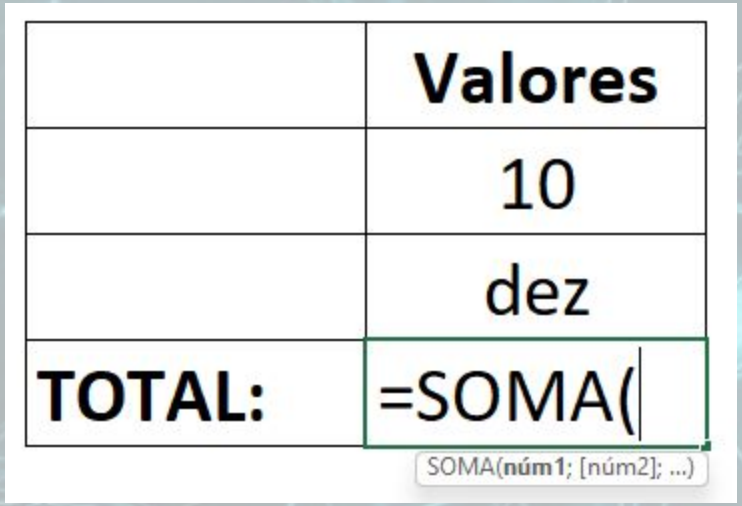
Atividade

Monte o banco de dados de um site do seu interesse, ex: um site de vendas de produtos(do seu interesse), um site de notícias. Lembre-se de criar as relações entre as tabelas e de informar qual o tipo dessa relação.

Tópico 1.3: Tipos de Dados e Restrições

Tipos de Dados

O banco de dados é "chato". Ele precisa saber exatamente que tipo de informação vai em cada gaveta (coluna) para economizar espaço e evitar erros.



	Valores
	10
	dez
TOTAL:	=SOMA(

SOMA(**núm1**; [núm2]; ...)

Excel

Os 4 principais tipos de Dados

1. **Numérico:**
   - ➔ **INT:** Números inteiros (Idade, quantidade, IDs).
   - ➔ **DECIMAL ou FLOAT:** Números com vírgula (Preços, peso, altura).
     - ◆ FLOAT usasse com frequência pois é rápido e tem boa precisão.
     - ◆ DECIMAL serve para cálculos com finalidades financeiras onde possui extrema precisão.

Os 4 principais tipos de Dados

2. **Texto:**
   - ➔ **VARCHAR(tamanho):** O "coringa". Texto de tamanho variável (Nomes, e-mails, senhas). O "tamanho" é o limite de letras.
   - ➔ **TEXT:** Para textos muito longos (Bio do perfil, descrição de produto).

Os 4 principais tipos de Dados

3. **Booleano:**
   - ➔ **BOOLEAN ou TINYINT(1)**: Verdadeiro ou Falso (O produto está ativo? O usuário é VIP?).
     - ◆ BOOLEAN nada mais é do que o número 0 que representa False, o 1 representa True e o NULL que representa Vazio.
     - ◆ TINYINT são as palavras em inglês TINY (pequeno) e INT (inteiro). Ocupa 1 byte e suporta números inteiro de -128 a 127. O numeral 1 entre parênteses define que o Banco vai mostrar apenas 1 número, e dessa forma usamos 0 e 1, para informar False e True respectivamente.

Os 4 principais tipos de Dados

4. **Temporais:**
   - ➔ **DATE:** Apenas a data (DD-MM-AAAA).
   - ➔ **DATETIME** ou **TIMESTAMP**: Data e hora exata (Momento de uma postagem ou compra).
     - ◆ DATETIME é apenas a data e hora. **Ex:** 2026-01-08 20:01:12
     - ◆ TIMESTAMP é data, hora e fuso horário. **Ex:** 2026-01-08 20:01:12.186 -0300

Os 4 principais tipos de Dados

4. **Temporais:**

   TIMESTAMP é assolado pelo "Bug do Milênio dos 32 bits" ou “o problema de 2038”.

   O TIMESTAMP usa um inteiro de 32 bits (que seu limite é “2.147.483.647”) para contar os segundos desde 1º de janeiro de 1970. Em binário esse número é representado por: “01111111 11111111 11111111 11111111”, onde o primeiro número, o 0 (zero), informa que esse número é positivo. Se adicionar + 1 a esse número ele se torna: “10000000 00000000 00000000 00000000”, dessa forma o primeiro número sendo 1 (um) ele vira um número negativo, o relógio saltaria de Janeiro de 2038 para Dezembro de 1901 causando erros em sistemas antigos.

Os 4 principais tipos de Dados

4. **Temporais:**

   Felizmente a solução já vem sendo aplicada a vários Sistemas e Bancos de Dados, migrando o sistema de 32 bits para 64 bits, onde consegue armazenar datas por mais de 290 bilhões de anos (o que é cerca de 20 vezes a idade estimada do universo).

   O Google teve que atualizar o contador de visualizações do YouTube por causa desse número. O clipe "Gangnam Style" foi o primeiro a ultrapassar 2,1 bilhões de views. Quando chegou perto desse limite, o contador quase quebrou, forçando o YouTube a migrar para 64 bits, que permite mais de 9 quintilhões de visualizações.

Atividade

Crie duas tabelas de um sistema ao seu gosto, além de nomear as coluna informe o tipo do dado de cada uma delas.

Tópico 1.4: Restrições

Restrições

As Restrições (Constraints) são as "regras de etiqueta" do seu banco de dados. Elas servem para garantir que nenhum dado inválido ou "lixo" entre nas suas tabelas.

Sem as constraints, o banco de dados seria apenas um depósito de textos e números sem ordem. Com elas, ele se torna um sistema confiável.

Restrições

1. **NOT NULL (Não Nulo):**

   Esta é a mais simples e uma das mais importantes. Ela impede que uma coluna fique vazia.

   Uso: Se você tem uma tabela de Usuários, o nome e o email devem ser NOT NULL. Ninguém pode se cadastrar sem essas informações.

   O que acontece: Se você tentar salvar um registro sem esse valor, o banco rejeita a operação e retorna um erro.

Restrições

2. **UNIQUE (Único):**

   Garante que todos os valores em uma coluna sejam diferentes uns dos outros.

   Uso: O email ou o CPF de um usuário. Duas pessoas podem ter o mesmo nome, mas nunca o mesmo e-mail no mesmo sistema.

   Diferença da PK: Uma tabela só tem uma Chave Primária, mas pode ter várias colunas UNIQUE. Além disso, colunas UNIQUE podem aceitar o valor NULL (dependendo do banco), enquanto a PK nunca aceita.

Restrições

3. **DEFAULT (Padrão):**

   Não é exatamente uma restrição de "bloqueio", mas uma regra de preenchimento. Se você não fornecer um valor, o banco coloca o valor padrão para você.

   Exemplo: status VARCHAR(10) DEFAULT 'Ativo'. Se você cadastrar o usuário e não disser o status, ele nasce como 'Ativo'.

Restrições

4. **AUTO INCREMENT:**

   Tem o trabalho de pegar o último valor definido, acrescentar + 1 e definir como o novo valor. Ex: Se o último livro cadastrado tem o ID 10 e você esta cadastrando um novo livro, ele vai gravar o novo livro com ID 11 sem que você precise passar esse valor manualmente.

   Utilizando esse parâmetro o ID nunca é reaproveitado, mesmo que você apague a linha 10, o próximo ID será 11, mantendo o histórico íntegro.

Atividade

Crie três tabelas de um sistema ao seu gosto, liste as colunas de cada tabela e informe se elas precisam ter uma dessas restrições.

Módulo 2:

Construção do Banco de Dados

Tópico 2.1: Conhecendo a Arquitetura

Divisões do Banco de Dados

O Banco de Dados tem 4 principais divisões, ou partes, como preferir. Cada uma é responsável por uma parte que é requisito de outra parte. Tornando elas dependentes uma das outras.

Sendo elas, DBMS, DDL, DML e DQL.

DBMS (Database Management System)

Ele é o Sistema de Gerenciamento de Banco de Dados, é um software que fica entre o usuário/aplicação e os dados físicos (os arquivos salvos no HD ou SSD). Ele desempenha quatro funções principais:

- **Criação:** Permite definir a estrutura dos dados (tabelas, colunas, tipos).
- **Manipulação:** Permite inserir, ler, atualizar e apagar dados (o famoso CRUD).
- **Segurança:** Controla quem pode ver o quê. Nem todo usuário do sistema pode ver a tabela de salários, por exemplo.
- **Integridade:** Garante que as regras (como as Constraints que vimos antes) sejam seguidas. Ele não deixa você apagar um cliente que tem dívidas, se houver uma regra para isso.

DBMS (Database Management System)

Existem dois tipos, Relacionais(SQL) e Não Relacionais(NoSQL).

- **SQL:** (Esse será o foco na Disciplina)
  - MySQL, PostgreSQL, Oracle, SQL Server, SQLite.
- **NoSQL:**
  - MongoDB, Redis, Neo4j, Cassandra.

DDL (Data Definition Language)

Tradução: Linguagem de Definição de Dados.

Conceito: É a parte do SQL responsável por criar, alterar ou excluir a estrutura dos objetos no banco de dados. Ela mexe com o "esqueleto" (tabelas, colunas, índices, restrições).

Foco: O esquema (Schema).

DML (Data Manipulation Language)

Tradução: Linguagem de Manipulação de Dados.

Conceito: É o conjunto de comandos usados para interagir com os dados que residem dentro das tabelas criadas pelo DDL. Com o DML, você altera o conteúdo, mas a estrutura da tabela permanece a mesma.

Foco: A informação (Linhas/Registros).

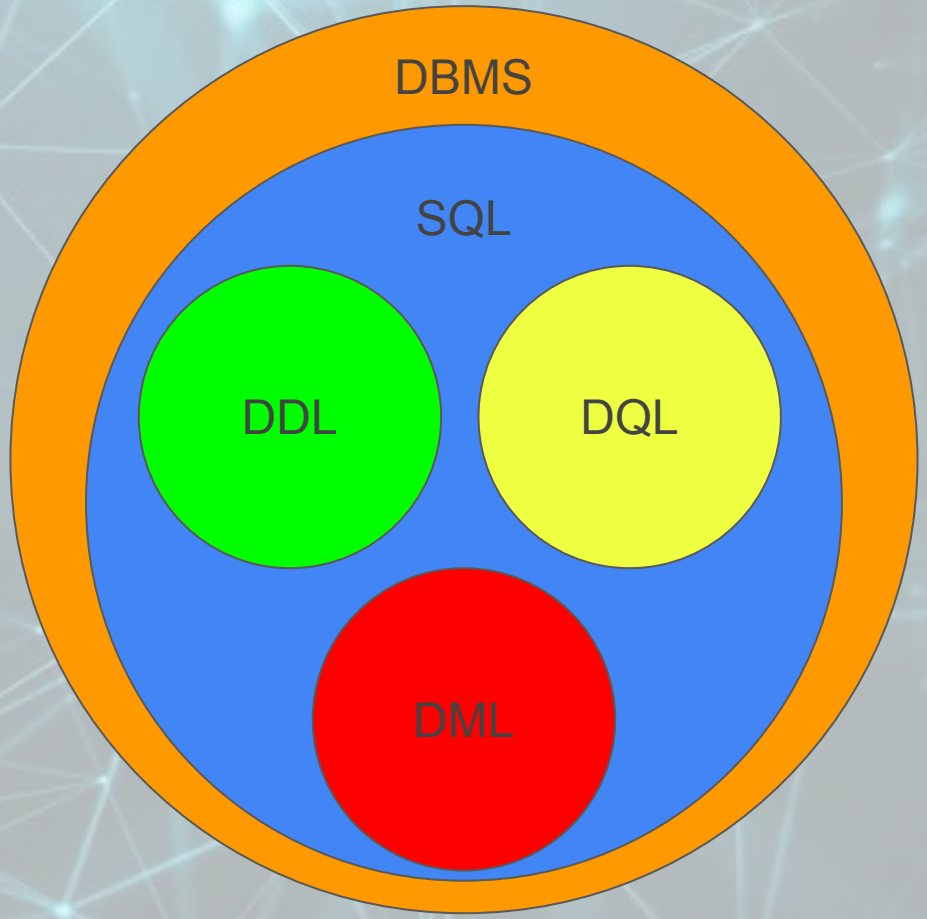
DQL (Data Query Language)

Tradução: Linguagem de Consulta de Dados.

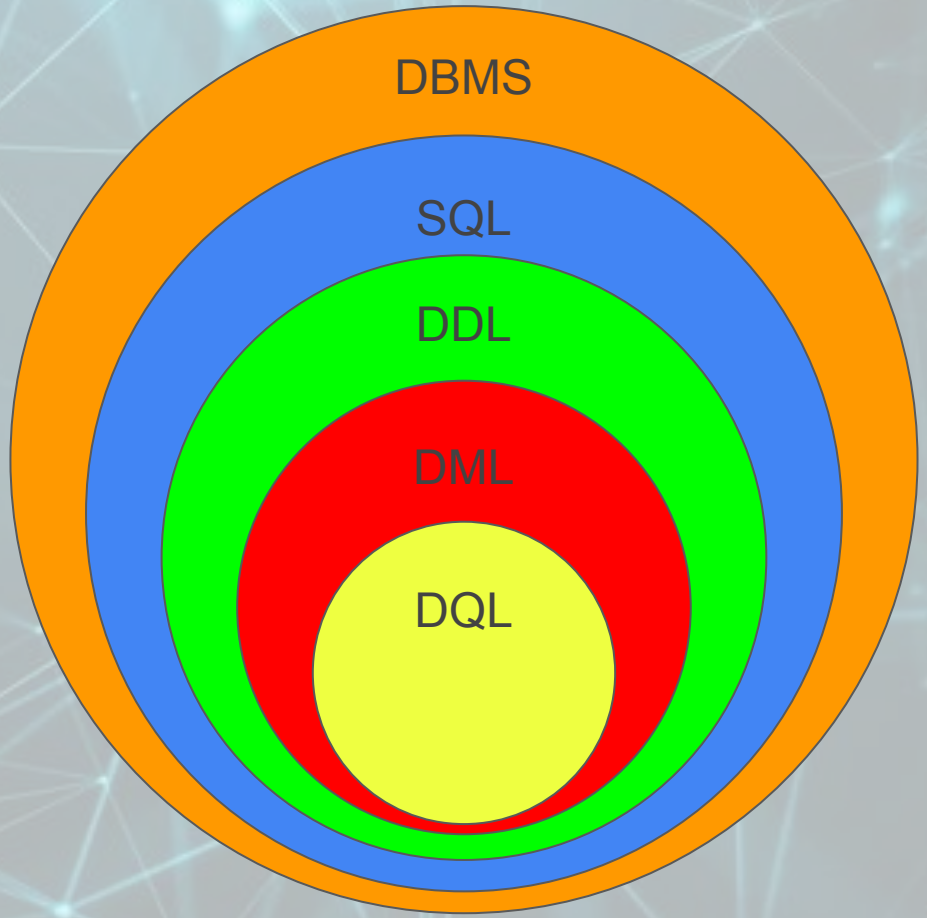
Conceito: É a parte do SQL focada exclusivamente na leitura e recuperação de informações. O DQL não altera a estrutura nem os dados; ele apenas "tira uma foto" do que está lá e mostra para você.

Foco: A visualização e filtragem.

Representação pelo ângulo de posição das camadas



Representação pelo ângulo de dependência de existência



Tópico 2.2: Configurando o ambiente

Instalando e configurando o DBMS (SQLite)

DBMS a ser utilizado: SQLite

Programa necessário: DBeaver

Instalando e configurando o DBMS (SQLite)

Após baixar e instalar o DBeaver:

1. Abra o DBeaver;
2. No canto superior esquerdo clique no ícone de tomada azul (Nova conexão);
3. Na listagem de Banco de Dados selecione o SQLite e clique em “Avançar”;
4. No campo “Caminho” clique em “Criar …”, na janela que abriu navegue até a pasta onde deseja salvar o arquivo do Banco. Digite o nome definido acrescido da extensão “.db” no final, ex: “meu_banco.db”, clique em salvar depois em concluir;
5. No menu lateral esquerdo, no Navegador de Banco de dados, dê um duplo clique no Banco recém criado;
6. Aparecerá um aviso para baixar o driver para o Banco, clique em “download”.

Instalando e configurando o DBMS (SQLite)

Com o banco criado e conectado, primeiros passos para escrever os comandos em SQL:

1. No menu lateral esquerdo onde está listado o Banco, dê um clique com o botão direito do mouse no Banco;
2. Na lista de opções que abriu, clique na primeira opção, “Editor SQL”, depois clique em “Novo script SQL”;
3. No editor que se abriu digite os comandos SQL, caso tenha rodado um comando de criação de tabela ou de colunas basta selecionar o Banco que esta trabalhando e clicar em F5 para atualizar o menu lateral esquerdo onde aparecerá a tabela dentro da conexão do Banco recém criado;
4. O arquivo “.db” que você criou é todo seu Banco, pode levá-lo para onde quiser.

Tópico 2.3: Usando os comandos DDL

Tipos de Dados no SQLite

Antes de programar, saiba que no SQLite:

- **TEXT:** Para texto e números formatados como texto.
- **INTEGER:** Para números inteiros.
- **REAL:** Para números decimais.
- **BLOB:** Para imagens e arquivos.

Tipos gerais	O SQLite armazena como...
INT, INTEGER, TINYINT	INTEGER
CHAR, VARCHAR, TEXT, CLOB	TEXT
FLOAT, REAL, DOUBLE	REAL
BLOB (arquivos, fotos)	BLOB
DATE, DATETIME, JSON, ENUM	TEXT (geralmente)

DDL (Data Definition Language)

Comandos principais:

1. **CREATE:** Cria uma nova tabela.

```
CREATE TABLE clientes ();
```

Sintaxe

Entretanto esse comando não vai rodar dessa forma, pois precisa das instruções das colunas.

DDL (Data Definition Language)

Comandos principais:

1. **CREATE:** Cria uma nova tabela.

Exemplos de criação das colunas.

```
CREATE TABLE clientes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT,
    idade INTEGER,
    altura REAL,
    foto_perfil BLOB
);
```

Sintaxe

DDL (Data Definition Language)

Comandos principais:

1. **CREATE:** Cria uma nova tabela.

Restrições(Constraints).

Impõe as restrições nos comandos DML.

Restrição	Objetivo	Exemplo de Uso
NOT NULL	Obrigatoriedade	nome TEXT NOT NULL
UNIQUE	Proibir duplicatas	email TEXT UNIQUE
DEFAULT	Valor padrão	estoque INTEGER DEFAULT 0
CHECK	Regra customizada	CHECK (salario > 0)
PRIMARY KEY	Identificador	id INTEGER PRIMARY KEY
AUTOINCREMENT	Autoincremento	id INTEGER PRIMARY KEY AUTOINCREMENT

DDL (Data Definition Language)

Comandos principais:

1. **CREATE:** Cria uma nova tabela.

Restrições(Constraints).
Impõe as restrições nos comandos DML.

```
CREATE TABLE produtos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL UNIQUE,
    preco REAL NOT NULL CHECK (preco > 0),
    estoque INTEGER DEFAULT 0
);
```

Sintaxe

DDL (Data Definition Language)

Comandos principais:

1. **CREATE:** Cria uma nova tabela.

Restrições(Constraints) AVANÇADO!
Impõe as restrições nos comandos DML.

```
CREATE TABLE usuarios (
    int INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT UNIQUE NOT NULL,
    idade INTEGER CHECK (idade >= 18 AND idade <= 30),
    saldo REAL DEFAULT 5.50,
    ativo INTEGER CHECK (ativo == 0 OR ativo == 1),
    estado_civil TEXT CHECK (estado_civil IN ('Solteiro', 'Casado', 'Divorciado'))
    bio TEXT CHECK (length(bio) <= 30)
);
```

Sintaxe

DDL (Data Definition Language)

Comandos principais:

2. **ALTER:** Modifica a estrutura de uma tabela (adiciona ou remove colunas).

```
ALTER TABLE clientes RENAME TO usuarios;
```

Sintaxe

DDL (Data Definition Language)

Comandos principais:

2. **ALTER:** Modifica a estrutura de uma tabela (adiciona ou remove colunas).

```sql
/* Renomeia a tabela */
ALTER TABLE usuarios RENAME TO clientes;
/* Adiciona coluna */
ALTER TABLE usuarios ADD COLUMN idade INTEGER;
/* Altera coluna */
ALTER TABLE usuarios RENAME COLUMN nome TO nome_completo;
/* Remove coluna */
ALTER TABLE usuarios DROP COLUMN foto_perfil;
```

Sintaxe

DDL (Data Definition Language)

Comandos principais:

3. **DROP:** Exclui a tabela inteira.

```
DROP TABLE clientes;
```

Sintaxe

DDL (Data Definition Language)

Para evitar um possível erro, faça uso do **IF EXISTS** e **IF NOT EXISTS** logo após a palavra TABLE:

```
CREATE TABLE IF NOT EXISTS clientes ();

ALTER TABLE clientes RENAME TO usuarios;

DROP TABLE IF EXISTS clientes;
```

ALTER não suporta EXISTS, portanto permanece como é.

Tópico 2.4: Usando os comandos DML

DML (Data Manipulation Language)

Comandos principais:

1. **INSERT:** Adiciona novos dados/registros/linhas a uma tabela.

```
INSERT INTO usuarios (nome) VALUES ("Ramon");
```

Sintaxe

DML (Data Manipulation Language)

Comandos principais:

1. **INSERT:** Adiciona novos dados/registros/linhas a uma tabela.

Inserir dados de várias colunas

```
INSERT INTO usuarios (nome, idade, ativo) VALUES ("Ramon", 20, 1);
```

Sintaxe

DML (Data Manipulation Language)

Comandos principais:

2. **UPDATE:** Altera dados existentes.

```
UPDATE usuarios SET nome_completo = 'Lucas' WHERE id = 3;
```

Sintaxe

DML (Data Manipulation Language)

Comandos principais:

2. **UPDATE:** Altera dados existentes.

Filtro duplo para correspondência

```
UPDATE usuarios SET nome_completo = 'Lucas' WHERE idade = 30 AND ativo = 1;
```

Sintaxe

DML (Data Manipulation Language)

Comandos principais:

3. **DELETE:** Remove linhas específicas de dados.

```
DELETE FROM usuarios WHERE id = 8;
```

Sintaxe

DML (Data Manipulation Language)

Comandos principais:

3. **DELETE:** Remove linhas específicas de dados.

Filtro duplo para correspondência

```
DELETE FROM usuarios WHERE nome_completo = 'Lucas' AND idade = 27;
```

Sintaxe

Tópico 2.5: Usando os comandos DQL

DQL (Data Query Language)

Comando principal:

- **SELECT:** Recupera dados do banco, permitindo filtrar, ordenar e agrupar.

Retorna todos os registros da tabela

```
SELECT * FROM usuarios;
```

Sintaxe

DQL (Data Query Language)

Comando principal:

- **SELECT:** Recupera dados do banco, permitindo filtrar, ordenar e agrupar.

Define quais colunas da tabela retornam

```
SELECT nome_completo, idade, ativo FROM usuarios;
```

Sintaxe

DQL (Data Query Language)

Comando principal:

- **SELECT:** Recupera dados do banco, permitindo filtrar, ordenar e agrupar.

Aplica filtro na busca

```
SELECT * FROM usuarios WHERE id = 2;
```

Sintaxe

Módulo 3:

Criando relacionamentos

Tópico 3.1: Um para muitos (1:N)

Relacionamento (1:N)

É o relacionamento mais comum. Uma pessoa pode fazer vários pedidos, mas um pedido só pode ser realizado por uma única pessoa.

```sql
CREATE TABLE usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL
);

CREATE TABLE livros (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    titulo TEXT NOT NULL,
    valor REAL NOT NULL,
    usuario_id INTEGER,
    FOREIGN KEY (usuario_id) REFERENCES usuarios (id)
);
```

Sintaxe

Tópico 3.2: Um para um (1:1)

Relacionamento (1:1)

É quando um registro da Tabela A só pode se relacionar com um registro da Tabela B, e vice-versa. É como a relação entre uma pessoa e seu CPF.

```sql
CREATE TABLE usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL
);

CREATE TABLE livros (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    titulo TEXT NOT NULL,
    valor REAL NOT NULL,
    usuario_id INTEGER UNIQUE,  /* <- Adiciona restrição UNIQUE */
    FOREIGN KEY (usuario_id) REFERENCES usuarios (id)
);
```

Sintaxe

Tópico 3.3: Muitos para muitos (N:N)

Relacionamento (N:N)

É quando vários registros da Tabela A podem se relacionar com vários registros da Tabela B. Um Aluno pode estar matriculado em várias Disciplinas, e uma Disciplina pode ter vários Alunos matriculados nela.

```sql
CREATE TABLE usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL
);

CREATE TABLE livros (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    titulo TEXT NOT NULL,
    valor REAL NOT NULL
);

CREATE TABLE usuario_livro (
    usuario_id INTEGER,
    livro_id INTEGER,
    FOREIGN KEY (usuario_id) REFERENCES usuarios (id),
    FOREIGN KEY (livro_id) REFERENCES livros (id)
);
```

Sintaxe

Tópico 3.4: ON DELETE

ON DELETE

O “ON DELETE” é uma cláusula que define o que o banco de dados deve fazer com os registros "filhos" quando o registro "pai" (o registro principal) é deletado. Se eu apagar o usuário “Ramon”, o que acontecerá com os pedidos que ele realizou? Ele decide o que acontecerá.

Opção	O que acontece com o Filho?	Quando usar?
CASCADE	Morre junto com o pai.	Dados dependentes (Itens de um pedido).
SET NULL	Continua vivo, mas sem dono.	Dados que podem existir sozinhos (Produtos).
RESTRICT	O pai não pode ser deletado.	Segurança máxima (Contas bancárias, faturas).

ON DELETE

- **Método:** CASCADE

  É o mais comum. Se você deletar o pai, todos os filhos são deletados automaticamente.

  - **Exemplo:** Se você deletar a conta de um usuário, todos os itens no carrinho de compras dele também são apagados.
  - **Vantagem:** Limpa o banco automaticamente, evitando dados inúteis.

ON DELETE

- **Método:** SET NULL

  Se você deletar o pai, o campo da chave estrangeira nos registros filhos vira NULL (vazio).

  - **Exemplo:** Você tem uma tabela de Produtos ligados a uma Categoria. Se você deletar a categoria "Promoção", os produtos não somem, eles apenas ficam "Sem Categoria".
  - **Requisito:** A coluna não pode ser NOT NULL.

ON DELETE

- **Método:** RESTRICT

O banco impede que você delete o pai enquanto houver filhos ligados a ele.

- **Exemplo:** Você não pode deletar um Cliente se ele tiver uma Fatura em aberto. Você precisa deletar (ou mover) as faturas primeiro para depois conseguir apagar o cliente.
- **Vantagem:** É a opção mais segura para evitar perda acidental de dados importantes.

ON DELETE

- **Como usar:** Sempre usar na chave estrangeira. Antes de passar o valor sempre colocar o nome da cláusula (ON DELETE) antes.

```
FOREIGN KEY (usuario_id) REFERENCES usuarios (id) ON DELETE CASCADE
FOREIGN KEY (usuario_id) REFERENCES usuarios (id) ON DELETE SET NULL
FOREIGN KEY (usuario_id) REFERENCES usuarios (id) ON DELETE RESTRICT
```

Sintaxe

ON DELETE

- **Exemplo:**

```sql
CREATE TABLE usuarios (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL
);

CREATE TABLE livros (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    titulo TEXT NOT NULL,
    valor REAL NOT NULL,
    usuario_id INTEGER,
    FOREIGN KEY (usuario_id) REFERENCES usuarios (id) ON DELETE CASCADE
);
```

Sintaxe